

Memory Is the Price: Evidence from the LLM Serving Market on the Economics of Mixture-of-Experts Inference

Michael J. Yuan, PhD*

Ju Long, PhD†

Working paper · Draft of July 2026

Data collected 2026-07-01

Abstract

Frontier language models have shifted decisively to sparse Mixture-of-Experts (MoE) architectures, which activate only a small fraction of their parameters per token and are widely claimed to cost proportionally less to serve. This claim is hard to check directly, because the production systems behind commercial inference endpoints are undisclosed: providers publish per-token prices, but almost never the cluster scale, parallelism, served precision, batch size, utilization, or margin of any endpoint. The serving configuration is thus a latent variable, and posted prices are its observable proxy: in a competitive market, sustained price structure identifies cost structure. From OpenRouter’s public endpoints API we assemble a panel of 121 provider–model price observations covering 18 open-weight models (12 MoE, 6 dense), joined to architecture parameters verified against each model’s HuggingFace configuration. The prices contradict the claim. A hedonic log-log regression of median output price on active parameters and the sparsity ratio yields $R^2 = 0.87$ with a sparsity elasticity of 0.554 ($t = 8.3$), which is 83% of the active-parameter elasticity: MoE models are billed most of the way toward their total memory footprint, and customers keep almost none of the advertised compute discount. The structure of the market explains why. Realizing MoE’s compute savings requires expert-parallel decoding pools far larger than a single node, so the price floor of every large-MoE panel is set by a vertically integrated “neocloud,” hyperscalers list identical models at 2–5 $\times$ that floor, and provider participation collapses with model footprint, which compresses the observed price dispersion through selection. We conclude that MoE serving economics are governed by memory capacity and bandwidth rather than FLOPS, and that the hardware suited to them is decode-oriented: large commodity memory, modest compute, moderate bandwidth.

1 Introduction

Between 2024 and 2026, the architecture of frontier open-weight language models changed decisively. Mixtral 8x7B (December 2023) reintroduced the sparse Mixture-of-Experts (MoE) design to open models with 8 experts per layer, of which 2 are activated per token. DeepSeek-V3 and R1 (December 2024 / January 2025) scaled the recipe to 671 billion total parameters with only 37 billion active per token (256 routed experts, top-8 routing) [1]. Within eighteen months, essentially every major open-weight release followed: Llama 4 Scout and Maverick (16 and 128 experts, top-1) [2, 3], Qwen3-235B (128 experts, top-8) [4], Kimi K2 (one trillion parameters, 384 experts) [5], GPT-OSS-120B (128 experts,

*ByteFuture Inc.

†Texas State University

top-4) [6], GLM-4.5 (160 experts) [7], and MiniMax M2 (256 experts) [8]. The sparsity ratio (total to active parameters) has grown from $\sim 4\times$ in Mixtral to $18\text{--}31\times$ in the current frontier. Meanwhile the largest open *dense* models have retreated: Llama-3.1-405B, the largest dense open release, is effectively delisted from the commercial serving market as of mid-2026.

Sparsity is marketed as an efficiency technology. The paper that popularized the design promised “outrageous numbers of parameters — but a constant computational cost” [9], and on compute-based reasoning a 671B-parameter model that computes with only 37B parameters per token should cost about as much to serve as a 37B dense model [10, 11]. Market prices do not bear this out. At the time of our data collection, DeepSeek R1 (37B active) commanded a median output price of \$2.39 per million tokens across providers, while Llama-3.3-70B, a dense model with *twice* R1’s active parameters, cost \$0.71. Per unit of active compute, the MoE model is roughly seven times more expensive. A second irregularity compounds the first: prices for the *same* model differ substantially across providers (Microsoft Azure and Amazon Bedrock list DeepSeek R1 at \$5.40 per million output tokens while the cheapest specialized GPU cloud lists it at \$2.15 [12, 13]), and the set of providers willing to serve a model at all shrinks sharply as models grow.

These patterns cannot be explained from provider disclosures, because there are almost none. The production systems behind commercial inference endpoints are, with rare exceptions, undisclosed. Hyperscaler model catalogs publish prices, context windows, regions, and quotas, but not hardware, served precision, parallelism, or cluster scale; Google’s managed-Llama announcement states simply that “the underlying infrastructure, scaling, and management costs are incorporated into the API usage price” [14]. On OpenRouter, the marketplace whose data we use, the per-endpoint schema contains no field for hardware or deployment topology, and even the self-reported quantization tag is frequently “unknown” [15]. The opacity extends to the served model itself: a black-box audit found that 11 of 31 commercial Llama endpoints produced output distributions deviating from Meta’s reference weights, because providers “may quantize, watermark, or finetune the underlying model... possibly without notifying users” [16].

The secrecy is not uniform, and the exceptions delimit what is actually hidden. The serving software stack is open source (vLLM, SGLang); the techniques for efficient MoE serving are published; and DeepSeek, a model vendor with an evident strategic interest in commoditizing the serving of its own models, has disclosed its production system in detail, down to its EP144 decode topology and a theoretical 545% margin [17], a recipe third parties have publicly reproduced [18]. What remains hidden is the mapping from any *particular* commercial endpoint to its deployment: which provider runs expert parallelism at what scale, at what batch size and utilization, at what unit cost. Engineering blogs occasionally reveal fragments (Cloudflare, for instance, has described the multi-node system behind its largest models [19]), but for the roughly 40 providers in our panel, the economically decisive parameters are unobservable.

The one thing every provider must publish is a price. This turns an information-asymmetry problem into an inference problem with a classical structure. The serving configuration behind an endpoint (cluster scale, parallelism degree, batch size, served precision, utilization) is the provider’s private information; the observer sees only the posted price. But prices are the market’s mechanism for aggregating dispersed private information [20], and open-weight serving approximates the textbook competitive case: the weights are free, the software is open source, entry is unrestricted, and dozens of providers quote side by side on a public marketplace. Competition disciplines posted prices toward marginal cost, and marginal cost is a function of the serving technology chosen. Sustained price structure therefore serves as an observable proxy for the latent cost structure and, through it, for the latent hardware configuration that generates that cost structure. Our empirical strategy follows the hedonic-pricing tradition [21]: we

regress prices on the product characteristics that drive resource consumption (active parameters; total memory footprint) and read the estimated coefficients as implicit prices, the market’s marginal valuation of each underlying resource. The industry’s claim that sparsity’s compute savings pass through into proportionally cheaper serving then becomes a testable restriction on hedonic coefficients (H1), and the question of what serving scale is required to capture those compute savings becomes a prediction about which provider class posts each panel’s price floor, who participates in each market, and how endogenous participation truncates the observed price distribution (H2).

We are explicit about the limits of this inversion. Price is a low-dimensional proxy: it cannot reconstruct any single endpoint’s configuration, and an individual quote confounds cost with strategy (loss leaders, price discrimination, enterprise positioning). Our inferences accordingly rest on moments of the price distribution that are difficult to generate strategically: elasticities estimated across 18 models, price floors replicated across seven independent panels, and participation patterns across ~ 40 providers.

Three strands of prior work frame our contribution. Bottom-up cost models compute what MoE serving *should* cost under optimal deployment [22, 23] but do not test market prices. Black-box measurement studies characterize hosted endpoints behaviorally: Gao et al. detect undisclosed weight modifications from outputs [16], and Li et al. document that hosted open-weight APIs are heterogeneous services whose listed prices are their most anchored attribute, observing descriptively, without a regression, that neither total nor active parameter count suffices to explain price [24]. Closest to our first hypothesis, Scher compared five MoE models against a dense price trendline in early 2025 and observed informally that MoE prices sit “somewhere between their active and total parameter count, perhaps closer to the total parameter count” [25]. We formalize that observation into an identified quantity. To our knowledge, this paper provides the first hedonic estimate of the sparsity elasticity of price (§4.1); the first evidence that the price floors for large MoE models are set exclusively by massively parallel providers (§4.2); and the first documentation that provider participation, and through selection measured price dispersion, is gated by model memory footprint (§4.2–4.3). The concluding section draws out the hardware implication.

2 Background and hypotheses

2.1 Background: the cost structure of decoding

Transformer inference has two phases. *Prefill* processes the prompt in parallel and is compute-bound. *Decode* generates output tokens one at a time; each step must read the model’s weights from memory while performing only ~ 2 floating-point operations per parameter read, making it memory-bound on all practical hardware. Output-token prices therefore reflect decode economics, and we study output prices throughout.

Two facts about the input markets frame our hypotheses. First, high-bandwidth memory (HBM) is the expensive and supply-constrained component of AI accelerators: HBM3E contract prices ran \$13–20/GB in 2025 (more than four times DDR5) and constitute 35–47% of an accelerator’s manufacturing cost [26, 27]; SK hynix, the leading supplier, sold out its capacity into 2026 [28]. Second, the number of GPUs a provider must deploy for a model is determined by the model’s *total* memory footprint, not by its compute: DeepSeek R1 at FP8 requires ~ 700 GB for weights, making an $8 \times H200$ node (1,128 GB) the standard minimal deployment [29]; Kimi K2 at FP8 fills the same node almost completely. Sparsity does not reduce this footprint: although only k of E experts fire per token, all E must reside in memory, because different tokens in a batch activate different experts [23].

A third fact concerns serving efficiency at scale. On a single 8-GPU node, a 256-expert model cannot be batched to compute efficiently: with top-8 routing, each expert receives only $K/32$ tokens per

step from a batch of K , far below the ~ 128 -token tile that saturates the grouped-GEMM kernels, so the node remains memory-bound at well under 1% model-FLOPS utilization at any feasible batch [30, 31]. Recovering efficiency requires *expert parallelism* (EP): spreading experts across many GPUs and aggregating tokens from thousands of concurrent users so that every expert sees full tiles. Published EP deployments include SGLang’s $96 \times \text{H100}$ system [18] and DeepSeek’s own production units of EP144 across 18 nodes [17].

2.2 Hypothesis 1: sparsity makes inference cheap (the compute-pricing view)

The proposition that MoE reduces serving cost by reducing compute has accompanied the architecture since its introduction, stated by its strongest proponents. Switch Transformers promised models “with outrageous numbers of parameters — but a constant computational cost” [9]. Mistral’s Mixtral $8 \times 7\text{B}$ announcement drew the pricing implication explicitly: the model “only uses 12.9B parameters per token” and “therefore, processes input and generates output at the same speed and for the same cost as a 12.9B model” [10]. Databricks made the same argument for DBRX, crediting “about half as many active parameters” for inference “up to 2x faster than LLaMA2-70B” [11]. Taken at face value, the claim yields a sharp prediction about market prices: a model’s output price is determined by its active parameters, and the sparsity ratio is free to the customer. We adopt this prediction as our first hypothesis. Formally, in the hedonic price regression

$$\log(\text{price}) = c_0 + c_1 \log(\text{active_params}) + c_2 \log(\text{total}/\text{active}) \quad (1)$$

the coefficients are implicit prices in Rosen’s sense [21]: c_1 is the market’s marginal valuation of active compute, and c_2 its valuation of resident-but-inactive parameters, the capacity that must be held in memory although it does not fire. *H1 (compute pricing)* predicts $c_2 = 0$. The alternative follows from §2.1: if memory rather than compute is the first-order cost of serving (capital tied up in HBM scales with bytes held, and decode throughput is limited by bytes streamed), then prices should track the *total* footprint, implying $c_2 \approx c_1$ (a $671\text{B}/37\text{B}$ MoE is billed like a 671B dense model). The ratio c_2/c_1 measures how much of the advertised sparsity discount the market actually passes to customers; under H1 it is zero.

2.3 Hypothesis 2: capturing the compute savings requires scale

While H1 concerns the composition of serving cost (which resource the market prices), our second hypothesis concerns scale economies: the minimum efficient scale at which a provider can realize the compute savings of sparsity. The mechanism of §2.1 implies that these savings, whatever share of them reaches customers, are only captured when enough concurrent traffic is aggregated that every expert sees full tiles, and that requires dedicated, disaggregated decoding infrastructure far larger than a single node. The serving-systems literature has converged on exactly this design. Splitwise showed that splitting prefill and decode onto separate machine pools yields $1.4 \times$ throughput at 20% lower cost, observing that decode “does not require the compute capability of the latest GPUs” [32]; DistServe disaggregates the phases onto different GPUs for up to $7.4 \times$ goodput [33]; and Moonshot’s production platform for Kimi, Mooncake, “separates the prefill and decoding clusters” outright [34]. For large MoE, the decode pool must additionally be expert-parallel, spread across tens to hundreds of GPUs [18, 17, 35]. The economic consequence is H2: the ability to serve large MoE at competitive prices is confined to providers who operate such pools and can amortize them across massive request volume. This predicts three observable market patterns: (a) the *price floor* for each large-MoE model should be set by specialized high-volume

GPU clouds (“neoclouds”) rather than by general-purpose clouds; (b) general-purpose clouds, whose business does not justify dedicated EP pools per open model, should price visibly above the floor; and (c) providers that cannot justify EP infrastructure at all (edge networks, boutique hosts, and by extension on-premises operators) should not offer large MoE models, exiting rather than listing at uncompetitive prices. A corollary concerns price *dispersion*: because exit removes the high-cost providers from observation, the measured spread of prices for large MoE should be *compressed* relative to dense models that any provider can serve.

3 Method and data

3.1 Price panel

We collected per-provider input and output prices for 27 candidate open-weight models on 2026-07-01 from OpenRouter’s public endpoints API (`openrouter.ai/api/v1/models/<slug>/endpoints`), the data source underlying its public model pages, cross-checked against Artificial Analysis provider pages and providers’ own published price lists [15, 36]. We excluded free tiers, zero-price rows, bring-your-own-key listings, and deprecated or deranked endpoints. Three independent collection agents gathered 123 raw observations; models with fewer than three qualifying providers were dropped. The final panel comprises **121 observations across 18 models (12 MoE spanning sparsity ratios E/k from 4 to 128, and 6 dense) from approximately 40 distinct providers**.

3.2 Architecture data

For each model we recorded total parameters, active parameters per token, expert count E , and routing top- k , verified directly against the model repository’s HuggingFace `config.json` (fetched the same day) [2, 3, 4, 5, 6, 7, 8, 37]. This verification surfaced one correction to commonly cited figures: GLM-4.5 has 160 routed experts (`n_routed_experts=160`), not 128. Dense models take `active = total` and $E/k = 1$.

3.3 Verification pass

A dedicated verification step re-fetched the five most anomalous price points. Two extreme but genuine values were retained (DeepInfra’s \$0.10/M output loss-leader on Qwen3-235B-A22B, confirmed on DeepInfra’s own price page [38]; Cloudflare’s \$2.25/M on Llama-3.3-70B); one duplicate listing was deduplicated (DeepInfra’s separate fp8/bf16 endpoints for Llama-3.1-8B); one stale row was dropped (a delisted SiliconFlow Qwen2.5-7B price). The same pass tested whether wide within-model spreads were explained by quantization differences across providers; they were not: in the three widest panels, the minimum and maximum prices were both fp8 offerings.

3.4 Statistical methods

To test H1 we compute each model’s median output price across providers (robust to loss-leaders and premium listings) and fit the hedonic regression (1) by ordinary least squares ($n = 18$). To test H2 (§2.3) we identify each panel’s floor-setting provider and its class; compare hyperscaler and GPU-cloud medians within panels listed on both; relate provider participation to model footprint; and measure within-model dispersion (coefficient of variation of output price) for MoE versus dense panels, including

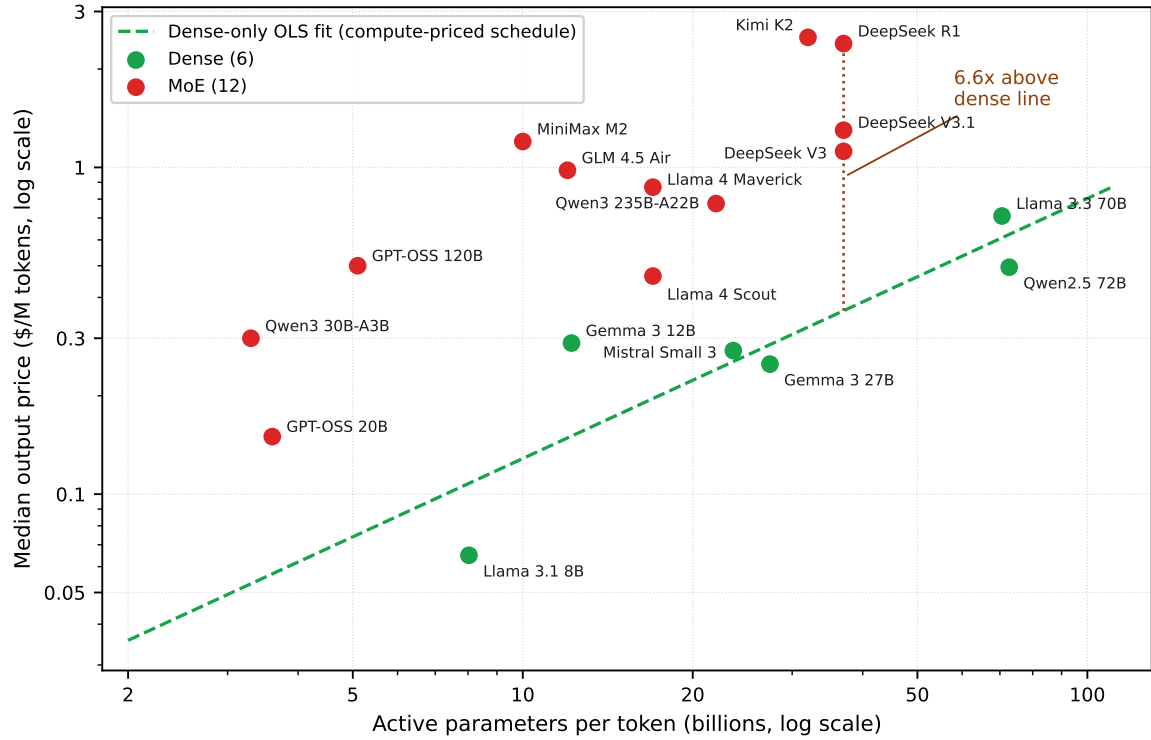


Figure 1: The figure plots median output price against active parameters per token for all 18 models on log-log axes, with dense models in green and MoE models in red. The dashed green line is the dense-only OLS fit, which is the price schedule a compute-priced market would apply to all models. Every MoE model sits above the line, and the vertical gap for DeepSeek R1 ($6.6\times$) is annotated.

a robustness split excluding hyperscaler and edge endpoints. All statistics are computed deterministically from the archived JSON with no model-generated numbers.

4 Results

4.1 H1 rejected: the market prices memory footprint, not active compute

Figure 1 plots median output price against active parameters on log-log axes. The six dense models trace a clean price line ($\log_{10} p = -1.687 + 0.796 \log_{10}(\text{active})$). Every MoE model sits above it, by $2.4\times$ to $9.3\times$, and the excess grows with the model’s sparsity ratio.

The regression quantifies the pattern (Table 1) and rejects H1 decisively. Compute pricing predicts $c_2 = 0$; the estimate is 0.554 with $t = 8.3$. The sparsity-ratio elasticity is large, precisely estimated, and close to the active-parameter elasticity: $c_2/c_1 = 0.83$. The sparsity ratio, which H1 predicts to be free, is in fact priced at 83% of the rate of active compute. The market bills MoE models most of the way toward their total memory footprint, so a 671B-total/37B-active model is billed roughly like a $\sim 430\text{B}$ -equivalent dense model rather than the 37B model its compute would suggest.

Table 2 reports the underlying panel. Per active parameter, MoE models cost $3\text{--}12\times$ more than dense models, and the per-model premium over the dense line tracks the sparsity ratio: Kimi K2 ($31\times$

Table 1: The table reports OLS estimates of the hedonic regression of log median output price on architecture ($n = 18$ models, $R^2 = 0.866$). H1 (compute pricing) predicts $c_2 = 0$, and pure footprint pricing predicts $c_2 = c_1$; the measured ratio is $c_2/c_1 = 0.83$.

Coefficient	Estimate	Std. err.	t
c_1 : log(active parameters)	+0.665	0.099	6.7
c_2 : log(total/active)	+0.554	0.067	8.3

Table 2: The table reports, for the 18-model panel, the median output price across providers (2026-07-01), the model architecture, the price per active-parameter-billion, and the multiple over the dense-only price line. Dense models, marked “—”, define the reference line.

Model	Median \$/M out	Active (B)	Total (B)	\$/active-B	$\times$ dense line	MoE
Kimi K2	2.50	32	1,000	0.078	7.7	yes
DeepSeek R1	2.39	37	671	0.065	6.6	yes
DeepSeek V3.1	1.30	37	671	0.035	3.6	yes
MiniMax M2	1.20	10	230	0.120	9.3	yes
DeepSeek V3	1.12	37	671	0.030	3.1	yes
GLM 4.5 Air	0.98	12	106	0.082	6.6	yes
Llama 4 Maverick	0.87	17	400	0.051	4.4	yes
Qwen3 235B-A22B	0.78	22	235	0.035	3.2	yes
GPT-OSS 120B	0.50	5.1	117	0.098	6.6	yes
Llama 4 Scout	0.47	17	109	0.027	2.4	yes
Qwen3 30B-A3B	0.30	3.3	30.5	0.091	5.6	yes
GPT-OSS 20B	0.15	3.6	21	0.042	2.6	yes
Llama 3.3 70B	0.71	70.6	70.6	0.010	—	no
Qwen2.5 72B	0.50	72.7	72.7	0.007	—	no
Gemma 3 12B	0.29	12.2	12.2	0.024	—	no
Mistral Small 3	0.28	23.6	23.6	0.012	—	no
Gemma 3 27B	0.25	27.4	27.4	0.009	—	no
Llama 3.1 8B	0.065	8.0	8.0	0.008	—	no

sparse) sits $7.7\times$ above the line, MiniMax M2 ($23\times$) $9.3\times$, GPT-OSS-120B ($23\times$) $6.6\times$, while the mildly sparse Llama-4 Scout and GPT-OSS-20B ($6\times$) sit $\sim 2.5\times$ above.

Interpretation. An 83% footprint weighting is what memory-dominated serving costs predict and is difficult to reconcile with compute-dominated costs. Every provider, including the cheapest, must hold and stream the full expert set regardless of how few parameters fire per token, and the market bills accordingly. The claims quoted in §2.2 are not reflected in market prices: a sparse model sells at a price set by the memory it occupies, not by its active parameters, and customers keep only a small share of the advertised sparsity discount.

4.2 H2 supported: the price floor belongs to massively parallel providers; others premium-price or exit

(a) *The cost gap between EP pools and small fleets is large.* Table 3 assembles the published deployment evidence. Per-GPU decode throughput on a $96\times$ H100 expert-parallel deployment is $\sim 36\times$ that of a

Table 3: The table compares decode throughput and cost across deployment scales. The $\sim 36\times$ per-GPU gap between the EP pool and the single node is the mechanism behind H2. NVIDIA’s Wide-EP measurements on GB200 NVL72 (EP32 delivers up to $1.8\times$ the per-GPU throughput of EP8) extend the pattern to current hardware [35].

Deployment (DeepSeek R1/V3 class)	tok/s per GPU	\$/M out tokens	Source
8 $\times$ H100, single node (AWQ 4-bit, vLLM)	~ 78	$\sim 7\text{--}10$ (market GPU rates)	[30]
96 $\times$ H100, PD-disagg. + large-scale EP (SGLang)	$\sim 2,787$	0.20 (self-reported)	[18]
DeepSeek production, EP144 decode (18-node H800)	$\sim 1,850$	<0.50 implied (545% margin at \$2.19)	[17]

Table 4: The table lists the price-floor provider for each of the seven large-MoE panels ($\geq 200\text{B}$ total parameters). No hyperscaler or edge provider sets a floor.

Panel (total params)	Floor \$/M out	Floor setter	Class
Kimi K2 (1,000B)	2.00	DeepInfra	neocloud
DeepSeek R1 (671B)	2.15	DeepInfra	neocloud
DeepSeek V3 (671B)	0.90	DeepInfra	neocloud
DeepSeek V3.1 (671B)	0.79	DeepInfra	neocloud
Llama 4 Maverick (400B)	0.60	DeepInfra	neocloud
Qwen3 235B-A22B (235B)	0.10	DeepInfra	neocloud
MiniMax M2 (230B)	1.20	MiniMax	model owner

single 8 $\times$ H100 node running the same model class; the EP operator’s self-reported cost is \$0.20 per million output tokens, versus $\sim \$7\text{--}10$ for the single node at market GPU rates. Because the retail floor was \$2.15–2.50, *no provider can sell at market price from an 8-GPU deployment*. The floor is only reachable with EP-scale pools.

(b) *The price floor is set by neoclouds; hyperscalers list at $2\text{--}5\times$ the floor.* In every one of the seven panels for models of $\geq 200\text{B}$ total parameters, the lowest price is posted by a specialized high-volume provider (Table 4): six of seven by DeepInfra alone, the seventh by the model’s owner. DeepInfra is the archetype of the class: roughly 30 employees, $\sim \$136\text{M}$ raised, GPUs owned outright (A100 through B300) in eight U.S. colocation sites, with published owned-hardware economics of $\sim \$1.98/\text{GPU-hour}$ against \$6.50 for equivalent rented public-cloud capacity, and an announced collaboration with NVIDIA on Dynamo disaggregated serving [12]. By contrast, Azure AI Foundry and Amazon Bedrock list DeepSeek R1 at \$1.35 input / \$5.40 output, $2.5\times$ the neocloud floor, and Azure lists DeepSeek V3 at \$4.56 against a \$0.90 floor, $5.1\times$, though the premium narrows to $1.4\text{--}2.2\times$ on V3.1. All are positioned explicitly as managed enterprise offerings [13]. Among the floor-setters, only SiliconFlow documents its serving architecture publicly, as co-author of the Huawei CloudMatrix-384 paper describing hierarchical expert parallelism on a 384-NPU supernode [39].

(c) *Providers without EP scale exit.* Market participation collapses with footprint, by provider class rather than by count alone. GPT-OSS-120B, which OpenAI states is “optimized to run on a single H100,” attracts 17 providers, including boutique and trusted-execution-environment specialists (Phala, DekaLLM, Mancer). DeepSeek R1/V3 at 671B attract 5–10 providers: exclusively large neoclouds plus premium hyperscaler listings, no boutiques. Kimi K2 at one trillion parameters attracts three. Cloudflare, an edge network with H100-class GPUs in more than 180 cities, offers only small dense models in our panel (Llama-3.1-8B and Llama-3.3-70B, in both cases at the *highest* price in the panel) and no

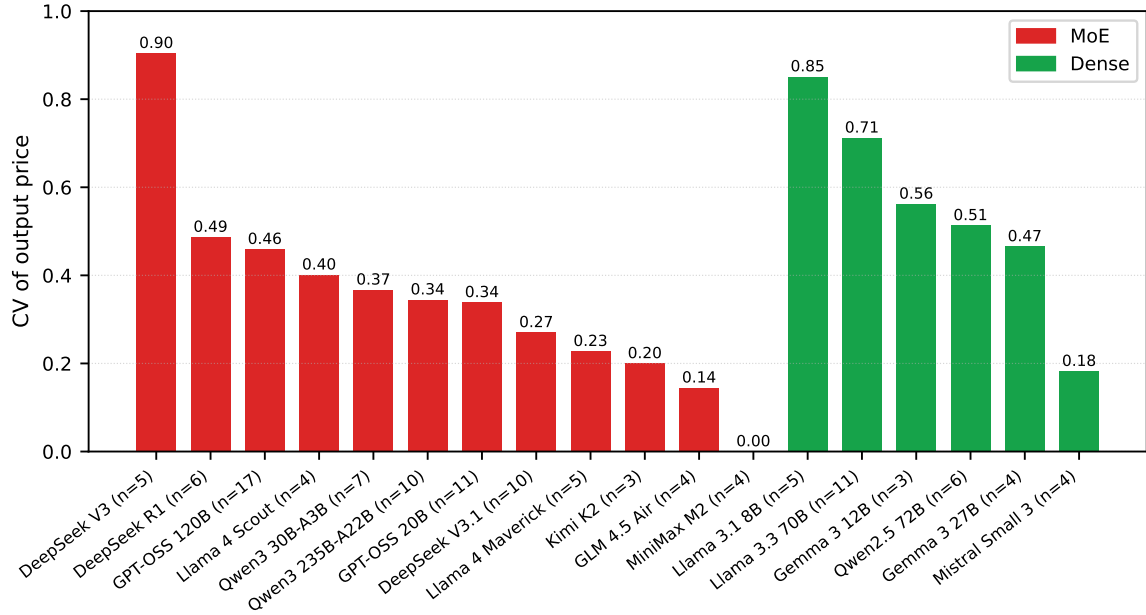


Figure 2: The figure shows within-model price dispersion, measured as the coefficient of variation of output price across providers, for MoE panels (red) versus dense panels (green).

671B-class model. The exception is instructive: in April 2026 Cloudflare began serving Kimi K2.x, but did so by building centralized multi-node serving with disaggregated prefill and expert parallelism in selected locations [19]. In effect, it constructed a small EP pool rather than serving the model at the edge. Entry into large-MoE serving requires becoming a neocloud.

4.3 Secondary result: price dispersion is compressed by selection

A natural first intuition, and our own initial prediction, is that the MoE cost mechanism should appear as *wider* price dispersion for MoE models, with under-scaled providers passing their higher costs through. The data show the opposite (Figure 2): MoE panels have *lower* within-model dispersion than dense panels (median coefficient of variation 0.34 versus 0.54; Spearman correlation between the sparsity ratio E/k and CV is -0.40 , and -0.54 when hyperscaler and edge endpoints are excluded).

The selection mechanism of H2(c) explains this. In the classical account, price dispersion for a homogeneous good reflects seller cost heterogeneity and buyer search frictions [40]; on that account, models that are harder to serve should disperse *more*. But participation here is endogenous: listing a model is a choice, and providers whose latent cost exceeds the competitive price decline to list rather than post a losing quote. The observed prices are therefore a selected sample [41]: the market truncates the upper tail of the latent cost distribution before it can appear as a price. Dense panels remain wide because any provider can serve a 70B dense model, so the full heterogeneity of provider cost structures (from bargain GPU clouds to premium edge networks at $5.7\text{--}7\times$ the floor) appears in the prices. What survives in large-MoE panels is instead a bimodal structure: a tight neocloud floor plus a hyperscaler premium tier at $2\text{--}5\times$. The compressed dispersion, initially surprising, corroborates H2 once selection is accounted for: the high prices that would widen the spread are never posted, because the providers who would have to post them do not enter.

5 Discussion and limitations

Our evidence is observational and market-level: we observe posted prices and public listings, not provider costs, contracts, or deployments. Five limitations qualify the interpretation of the results. For each, we state the limitation and the reason we believe the conclusions survive it.

- **Price is not cost.** List prices embed margins, loss-leader strategies, and enterprise positioning. The footprint elasticity establishes what the market *charges*; the inference to underlying cost is an interpretation, supported by the deployment-cost evidence of Table 3 (EP-pool cost \$0.20–0.50/M against floors of \$0.79–2.50) and by the competitive structure of open-weight serving, but LMSYS’s \$0.20/M and DeepSeek’s 545% margin figures are self-reported, not audited.
- **Inference from prices is indirect.** Provider opacity is the premise of our design (§1), but it also limits what we can claim: that the floor-setting neoclouds run large-scale expert parallelism is inferred from their economics and from the public availability of EP serving software, not observed; only SiliconFlow documents an EP deployment. No small provider states publicly why it declines to serve 671B-class models; the exit mechanism rests on documented economics rather than testimony. Our conclusions are consistency checks of the price structure against publicly reconstructable cost floors, not observations of provider internals.
- **Demand and quality are unobserved.** DeepSeek R1 commands $2.1\times$ the price of its architecture-identical sibling V3, a reasoning-model demand premium that the regression absorbs as noise. Popularity may also correlate with aggressive pricing.
- **The panel is a single snapshot.** It reflects one collection date (2026-07-01) in a market that reprices within weeks: DeepInfra’s own R1 price moved from the panel’s \$2.15 to \$2.50 between collection and verification, and an earlier $14\times$ R1 spread reported in a prior draft had collapsed to $\sim 2.5\times$ by collection time. The elasticities should be re-estimated longitudinally; the collection pipeline is automated.
- **The sample has limits.** OpenRouter covers the competitive open-weight market; hyperscaler-exclusive and closed models are underrepresented. The dense subsample is small ($n = 6$) and confined to $\leq 73\text{B}$, because larger dense models have left the market, which is itself evidence of footprint economics. Delivered throughput tiers are not controlled. Finally, capacity and bandwidth requirements both scale with total parameters, so our test cannot distinguish “priced by bytes held” from “priced by bytes streamed”; under either reading, the priced quantity is memory, not compute.

6 Conclusion: implications for inference hardware

Two results emerge from the serving market’s own prices. First, the industry’s original claim for sparsity, that an MoE model costs what its active compute costs [9, 10, 11], is rejected: the market bills Mixture-of-Experts models by the memory they occupy, not the compute they perform. The sparsity elasticity of price is 83% of the active-parameter elasticity, so a customer of a 671B/37B model pays approximately as if for a $\sim 430\text{B}$ dense model. Second, the ability to serve large MoE near marginal cost is confined to a handful of massively parallel providers. General-purpose clouds price $2\text{--}5\times$ above them, and everyone else (edge networks, boutique hosts, and by extension on-premises operators using the same GPU building blocks) either pays those premiums or exits.

Both results point to the same hardware mismatch. The mainstream GPU bundles enormous compute with scarce, expensive HBM; MoE decoding uses the memory fully and the compute hardly at all, except inside hyperscale expert-parallel pools that only a few operators can build. For everyone else, the economically rational serving device is the opposite shape: **large, cheap memory capacity** to hold the full expert set without buying stranded FLOPS; **modest compute**, since decode at realistic concurrency cannot exercise more; and **moderate bandwidth**, sufficient for interactive token rates and procurable from commodity supply chains rather than the allocation-constrained HBM market. Several shipping products approximate this design point, notably Apple’s Mac Studio (up to 512 GB unified LPDDR5) and NVIDIA’s DGX Spark (128 GB LPDDR5X), and purpose-built decode accelerators such as the Skymizer HTX-301 (384 GB commodity LPDDR on a 28 nm PCIe card; performance figures vendor-claimed pending independent benchmarks) instantiate it directly. The market prices measured here quantify the opportunity such hardware addresses: the gap between footprint-cheap serving cost and market price is widest for the most sparse frontier models (Kimi K2, MiniMax M2, GPT-OSS-120B), which is where the industry is heading.

Data and reproducibility. The raw data and analysis code are available at https://github.com/juntao/memory_is_the_price: the archived price panel (`pd_prices.json`, 123 raw rows; 121 qualifying observations), architecture data (`pd_arch.json`), derived statistics (`pd_dispersion.json`, `pd_level.json`), the analysis script (`analysis.py`, dependency-free Python: OLS via normal equations, Spearman by ranks), and the figure-generation script (`make_figures.py`). Running `analysis.py` reproduces every statistic in §4. All references below were verified by direct fetch at the time of writing; architecture parameters were verified against each repository’s `config.json`.

References

- [1] DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437. <https://arxiv.org/abs/2412.19437>
- [2] Meta. Llama-4-Scout-17B-16E-Instruct model repository ($E=16, k=1$). HuggingFace, fetched 2026-07-01. <https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E-Instruct>
- [3] Meta. Llama-4-Maverick-17B-128E-Instruct model repository ($E=128, k=1$). HuggingFace, fetched 2026-07-01. <https://huggingface.co/meta-llama/Llama-4-Maverick-17B-128E-Instruct>
- [4] Qwen Team. Qwen3-235B-A22B-Instruct-2507 model repository, `config.json` ($E=128, k=8$). HuggingFace, fetched 2026-07-01. <https://huggingface.co/Qwen/Qwen3-235B-A22B-Instruct-2507/raw/main/config.json>
- [5] Moonshot AI. Kimi-K2-Instruct model repository, `config.json` ($E=384, k=8$). HuggingFace, fetched 2026-07-01. <https://huggingface.co/moonshotai/Kimi-K2-Instruct/raw/main/config.json>
- [6] OpenAI. gpt-oss-120b model repository, `config.json` ($E=128, k=4$). HuggingFace, fetched 2026-07-01. <https://huggingface.co/openai/gpt-oss-120b/raw/main/config.json>
- [7] Z.ai. GLM-4.5 model repository, `config.json` (`n_routed_experts=160, k=8`). HuggingFace, fetched 2026-07-01. <https://huggingface.co/zai-org/GLM-4.5/raw/main/config.json>
- [8] MiniMax AI. MiniMax-M2 model repository ($E=256, k=8$; 230B total / 10B active). HuggingFace, fetched 2026-07-01. <https://huggingface.co/MiniMaxAI/MiniMax-M2>

- [9] Fedus, W., Zoph, B., & Shazeer, N. Switch Transformers: scaling to trillion parameter models with simple and efficient sparsity. arXiv:2101.03961, 2021. <https://arxiv.org/abs/2101.03961> (“with outrageous numbers of parameters — but a constant computational cost”).
- [10] Mistral AI. Mixtral of experts. 2023-12-11. <https://mistral.ai/news/mixtral-of-experts/> (“Mixtral has 46.7B total parameters but only uses 12.9B parameters per token. It, therefore, processes input and generates output at the same speed and for the same cost as a 12.9B model.”)
- [11] Databricks. Introducing DBRX: a new state-of-the-art open LLM. 2024-03-27. <https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm> (“thanks to having about half as many active parameters — DBRX inference throughput is up to 2x faster” than Llama2-70B).
- [12] DeepInfra. Series B announcement and DeepCluster owned-hardware pricing. <https://deepinfra.com/blog/deepinfra-series-b>; <https://deepinfra.com/deepcluster>
- [13] Microsoft Azure AI Foundry, DeepSeek model pricing. <https://azure.microsoft.com/en-us/pricing/details/ai-foundry-models/deepseek/>; Amazon Web Services, DeepSeek-R1 on Bedrock. <https://aws.amazon.com/blogs/aws/deepseek-r1-now-available-as-a-fully-managed-serverless-model-in-amazon-bedrock/>
- [14] Google. Llama 4 models generally available as managed API service on Vertex AI. Google Developers Blog, 2025-04-29. <https://developers.googleblog.com/en/llama-4-ga-maas-vertex-ai/> (“The underlying infrastructure, scaling, and management costs are incorporated into the API usage price.”). See also AWS Bedrock model card for DeepSeek-R1 (commercial terms only): <https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-deepseek-deepseek-r1.html>
- [15] OpenRouter. Public endpoints API, per-model provider listings. <https://openrouter.ai/api/v1/models/deepseek/deepseek-r1-0528/endpoints> (pattern identical for all 18 models). Accessed 2026-07-01; re-verified at writing.
- [16] Gao, I., Liang, P., & Guestrin, C. Model equality testing: which model is this API serving? ICLR 2025. arXiv:2410.20247. <https://arxiv.org/abs/2410.20247>
- [17] DeepSeek-AI. DeepSeek-V3/R1 inference system overview (Open Infra Index, Day 6). https://github.com/deepseek-ai/open-infra-index/blob/main/202502OpenSourceWeek/day_6_one_more_thing_deepseekV3R1_inference_system_overview.md
- [18] LMSYS. Deploying DeepSeek with PD disaggregation and large-scale expert parallelism on 96 H100 GPUs. <https://www.lmsys.org/blog/2025-05-05-large-scale-ep/>
- [19] Cloudflare. Serving very large models on Workers AI (Infire; disaggregated prefill and expert parallelism). <https://blog.cloudflare.com/workers-ai-large-models/>; model catalog: <https://developers.cloudflare.com/workers-ai/models/>
- [20] Hayek, F. A. The use of knowledge in society. *American Economic Review* 35(4), 519–530, 1945.
- [21] Rosen, S. Hedonic prices and implicit markets: product differentiation in pure competition. *Journal of Political Economy* 82(1), 34–55, 1974.
- [22] Erdil, E. Inference economics of language models. Epoch AI. arXiv:2506.04645, June 2025. <https://arxiv.org/abs/2506.04645>. Bottom-up cost model over arithmetic, memory-bandwidth, network, and latency constraints; no market-price analysis.

- [23] Tensor Economics. MoE inference economics from first principles. <https://www.tensoreconomics.com/p/moe-inference-economics-from-first>
- [24] Li, H., et al. When is the same model not the same service? A measurement study of hosted open-weight LLM APIs. arXiv:2605.02821, May 2026. <https://arxiv.org/abs/2605.02821>. §4.4 (“Parameter Scale Is Not a Sufficient Price Model”) raises the active-vs-total pricing question descriptively.
- [25] Scher, A. Observations about LLM inference pricing. MIRI Technical Governance Team blog, 2025-03-03. <https://techgov.intelligence.org/blog/observations-about-llm-inference-pricing>. Fits dense price–parameter regressions and compares five MoE models against the dense trendline.
- [26] TrendForce. HBM3e pricing and DDR5 premium. <https://www.trendforce.com/presscenter/news/20251029-12758.html>
- [27] Silicon Analysts. AI accelerator cost bridge: HBM share of manufacturing cost. <https://siliconanalysts.com/tools/cost-bridge>
- [28] TechSpot. SK hynix sells out semiconductor supply into 2026. <https://www.techspot.com/news/110058-sk-hynix-completely-sells-out-semiconductor-supply-ai.html>
- [29] HPC-AI Tech. DeepSeek R1 671B deployment guide (8×H200 reference configuration). <https://company.hpc-ai.com/blog/deepseek-r1-671b-deployment-how-to-maximize-performance>
- [30] dzhsurf. DeepSeek V3/R1 deployment benchmarks, 8×H100 (AWQ 4-bit, vLLM). <https://github.com/dzhsurf/deepseek-v3-r1-deploy-and-benchmarks>
- [31] DeepSeek-AI. DeepGEMM: grouped GEMM kernels (BLOCK_M = 128). <https://github.com/deepseek-ai/DeepGEMM>
- [32] Patel, P., Choukse, E., et al. (Microsoft Azure Research). Splitwise: efficient generative LLM inference using phase splitting. arXiv:2311.18677, 2023. <https://arxiv.org/abs/2311.18677>
- [33] Zhong, Y., et al. DistServe: disaggregating prefill and decoding for goodput-optimized large language model serving. OSDI 2024. arXiv:2401.09670. <https://arxiv.org/abs/2401.09670>
- [34] Qin, R., et al. (Moonshot AI). Mooncake: a KVCache-centric disaggregated architecture for LLM serving. arXiv:2407.00079, 2024. <https://arxiv.org/abs/2407.00079>
- [35] NVIDIA. Scaling large MoE models with wide expert parallelism on NVL72 rack-scale systems. <https://developer.nvidia.com/blog/scaling-large-moe-models-with-wide-expert-parallelism-on-nvl72-rack-scale-systems/>
- [36] Artificial Analysis. Provider comparison pages. <https://artificialanalysis.ai/models/deepseek-r1/providers>
- [37] DeepSeek-AI. DeepSeek-R1-0528 model repository, `config.json` ($E=256, k=8$). HuggingFace, fetched 2026-07-01. <https://huggingface.co/deepseek-ai/DeepSeek-R1-0528/raw/main/config.json>
- [38] DeepInfra. Qwen3-235B-A22B-Instruct-2507 price page (loss-leader verification). <https://deepinfra.com/Qwen/Qwen3-235B-A22B-Instruct-2507>

- [39] Huawei & SiliconFlow. Serving large language models on Huawei CloudMatrix384. arXiv:2506.12708. <https://arxiv.org/pdf/2506.12708>
- [40] Stigler, G. J. The economics of information. *Journal of Political Economy* 69(3), 213–225, 1961.
- [41] Heckman, J. J. Sample selection bias as a specification error. *Econometrica* 47(1), 153–161, 1979.